

GENERAL

GETTING TO GRIPS WITH SUPPORT VECTOR MACHINES: APPLICATION

Steve Kroon

Department of Statistics, University of Stellenbosch, Private Bag X1,
Matieland, South Africa; 7602

e-mail: *kroon@sun.ac.za*

and

Christian W. Omlin

Department of Mathematics and Computing Science, University of the South
Pacific, Suva, Fiji Islands

e-mail: *omlin@usp.ac.fj*

Key words: Classification; LIBSVM; primer; R; SVM; tutorial.

Summary: The support vector machine (SVM) is a technique for function estimation which was proposed in the early 1990s. This paper is a follow-up to the paper *Getting to grips with Support Vector Machines: Theory*, which gave an introductory overview of SVMs. This paper discusses issues arising when applying the SVM in practice, giving useful hints and suggestions for newcomers to the field. The focus is on the SVM classifier, although the support vector technique is also applicable to regression and other techniques.

1. Introduction

Since its introduction in Boser, Guyon and Vapnik (1992), the SVM has proven to be a versatile technique, able to effectively handle a wide variety of tasks.

This paper assumes familiarity with the content of its precursor in this issue, *Getting to grips with Support Vector Machines: Theory*, where the SVM was introduced in a number of stages: the separating hyperplane, the kernel trick, and introducing slack variables to handle inseparability.

STMA (2004) SUBJECT CLASSIFICATION: 04:160(04:900)

This paper focusses on applying SVMs. Some of the more common implementations of SVMs are briefly discussed in Section 2. Section 3 presents an example of the application of an SVM package to solve a simple classification problem.

2. Implementing SVMs

SVM algorithms are processor and memory intensive, and for this reason much research has focused on optimizing the algorithms' memory and processor usage. Thus the SVM packages available for download usually make use of custom implementations for solving the quadratic programming problem, employing various techniques to speed up the execution and reduce the memory requirements relative to a naive implementation using a traditional quadratic programming library.

A variety of SVM implementations and utilities are available. Well-known packages include:

- LIBSVM (Chang and Lin, 2000), arguably the most popular SVM implementation in practice. This library, which can perform classification (including multi-class problems), regression, and density estimation, is kept up to date with new developments in the SVM literature, and the authors are recognised experts in the algorithmic efficiency of SVMs. There is also a LIBSVMtools website providing various modifications of the core program. There is a link there from the LIBSVM website, which also provides manuals and introductions for the SVM novice. Another appealing feature of this library is that interfaces have been developed to it for use in the Matlab, R and S-Plus computing environments. Lin is also one of the proponents of the package *looms* (Lee and Lin, 2000), which trains SVMs and performs model selection using a leave-one-out cross-validation technique. Unfortunately, *looms* can currently only handle the Gaussian kernel.

- *SVM^{light}* by Joachims (1998), is an SVM classifier designed to handle large data sets. It provides the more common kernel functions and also allows custom specification of kernels. An additional feature is the ability to handle cost models (i.e. alternatives to the hinge loss function). Joachims has led the field in applying SVMs to text classification (Joachims, 1997), and his implementation is particularly suited to similar problems (high-dimensional input space with many zero or empty components in input vectors). The research of Collobert and Bengio (2000) yielded SVM_{Torch}. This is a regression equivalent of the *SVM^{light}* implementation. It is the SVM component of the Torch machine learning library from the Dalle Molle Institute for Perceptual Artificial Intelligence (<http://www.idiap.ch>). SVM_{Torch} II is now available, but note that file formats between the two versions are not compatible. A file converter is available on the SVM_{Torch} website. Another regression counterpart for *SVM^{light}* by Stefan Rüping is *mySVM*.
- The SVM Matlab toolbox by Gunn (1998) is also popular. Other Matlab toolboxes are available for SVM but this is the best known. It handles classification and regression, and can provide visual display of results for simple examples.

A Fortran library for SVM classification and regression is also available. Links to implementations (including all those mentioned above), libraries, and code for various optimizations are provided at <http://www.kernel-machines.org/software.html>.

3. Example

In this section, we will consider a simple problem, and its solution using an SVM. The software used is LIBSVM (version 2.4), and the example is done on the Linux operating system. This is primarily because the authors are more familiar with Linux. Furthermore, it has been observed that the Windows version of LIBSVM sometimes suffers unexplained delays in training on large data sets with certain settings (Steel, 2003). The first step in employing an SVM package is preparing the data for use by the package. This usually

entails replacing any text labels by numeric values. For example a classification problem with outcomes ‘malign’ and ‘benign’, would have, e.g. ‘malign’ replaced throughout by -1 , and ‘benign’ by 1 . Although the effect of scaling data inputs to the SVM is not completely understood — the effects depend on the exact kernel used — for most kernels it is recommended that continuous data components should be scaled to lie in $[0, 1]$ or $[-1, 1]$, or standardized to zero mean and unit variance. Finally, most SVM packages are very specific about the exact format in which the data file is stored. In the case of the R interface (available in the `e1071` package), these details are automatically handled (i.e. data formatting, converting labels to numbers, and scaling). The LIBSVM package provides a utility, `svm-scale` for the scaling, while file formatting must be done manually.

Once the data have been prepared, the SVM is trained, and the resulting model is stored in a file for future prediction purposes. For LIBSVM, the command used to do this is `svm-train`. After the model is generated, one can use the model file for predictions using the `svm-predict` program. For illustration, we consider Fisher’s well-known three-class problem of predicting which of three iris species a specimen belongs to, based on four measurements. The data set used here was obtained from the UCI data repository Blake and Merz (1998). The file `bezdekIris.data` should be used, since `iris.data` differs slightly from the data in Fisher’s groundbreaking 1936 article.

The original data set has the form

```
5.1, 3.5, 1.4, 0.2, Iris-setosa
4.9, 3.0, 1.4, 0.2, Iris-setosa
4.7, 3.2, 1.3, 0.2, Iris-setosa
4.6, 3.1, 1.5, 0.2, Iris-setosa
```

with 150 such examples. For use by the SVM, it must be converted into the form

```
1 1:5.1 2:3.5 3:1.4 4:0.2
1 1:4.9 2:3.0 3:1.4 4:0.2
1 1:4.7 2:3.2 3:1.3 4:0.2
1 1:4.6 2:3.1 3:1.5 4:0.2
```

Here the response has been put at the beginning of each line, and the row numbers have been deleted. One could now optionally use the `svm-scale` command to rescale this data between two extreme values. We have not done this, since the scaling properties of LIBSVM and its R interface are different: LIBSVM requires the user to provide upper and lower limits, and scales the data for each component to lie in the range, while the R interface scales the data to zero mean and unit variance prior to presenting the data to the SVM routine. R then stores the scaling factor, and scales future inputs similarly before using the SVM model to make predictions. This behaviour makes it more difficult to compare the results using scaling.

Suppose the data set is in the format above. We separate it into a training set consisting of the first 30 data points for each class, and use the remaining 20 of each class for testing our SVM. Suppose we put the training set in the file `iris.train` and our test data in `iris.test`. We can then run the command

```
./svm-train iris.txt iris.model
```

to generate our SVM model. This should generate the following output:

```
> ./svm-train iris.train iris.model
.*
optimization finished, #iter = 96
nu = 0.064753
```

```

obj = -2.011841, rho = 0.098439
nSV = 10, nBSV = 1
*
optimization finished, #iter = 41
nu = 0.060473
obj = -1.814402, rho = 0.205714
nSV = 9, nBSV = 0
*
optimization finished, #iter = 40
nu = 0.390381
obj = -16.413092, rho = 0.095269
nSV = 26, nBSV = 21
Total nSV = 36

```

LIBSVM makes use of pairwise coupling for multiclass problems so that 3 standard 2-class SVMs are trained. Here the dots and asterisk in the first line of each of the 3 portions are a progress indicator. This output tells us the optimum value `obj` of the objective function achieved, and the value of the bias term for each machine `rho`. It also provides the number of support vectors for each machine (10, 9 and 26 in this case), and at the end the total number of data points used as support vectors in at least one machine.

We can obtain even more information by investigating the model file, `iris.model`:

```

svm_type c_svc
kernel_type rbf
gamma 0.25
nr_class 3
total_sv 36

```

```
rho 0.0984391 0.205714 0.0952694
label 1 2 3
nr_sv 5 16 15
SV
0.667594162860 0.486460321085 1:4.4 2:2.9 3:1.4 4:0.2
0 0.436669350458 1:4.3 2:3 3:1.1 4:0.1
0.604855853135 0.891065574247 1:5.7 2:4.4 3:1.5 4:0.4
...
```

From the line saying SV downwards, the support vectors are listed, preceded by their associated Lagrange multipliers for each of the SVMs involving the class of that data point. Above this, are a number of settings. The first line specifies that the model is based on a traditional soft-margin classifier. The default value of C is 1. The next line specifies that a Gaussian kernel has been used, with a γ -value of $\frac{1}{4}$. The following lines specify that the model is for a 3-class problem, and that there are 36 support vectors. The values of the bias for each SVM is then provided, followed by the labels for the classes, and the number of support vectors belonging to each class, respectively. Note that the settings of the SVM type, and the values of C and γ were not specified when `svm-train` was run. There are a number of default values used, although they can be changed. More details can be found in the LIBSVM documentation, and a short listing can be obtained by just running `./svm-train` without any arguments.

Once the model file has been built we can evaluate its performance on a test set. Here we need to specify the model file, the file containing the test set, and a file for the output. Thus, running

```
./svm-predict iris.test iris.model iris.out
```

generates a file called `iris.out` containing the predictions, and outputs

```
> ./svm-predict iris.test iris.model iris.out
Accuracy = 96.6667% (58/60) (classification)
Mean squared error = 0.0333333 (regression)
Squared correlation coefficient = 0.950625 (regression)
```

The accuracy is then the classification rate on the test set. If one wanted to do predictions without access to the correct values, one still uses the program `svm-predict`, but the test file just does not contain the first column with the response variables. The accuracy quoted by LIBSVM in this case should be ignored. In both cases (with and without the response values), the predictions made by the SVM will be stored in `iris.out`.

We now focus on using the R computing environment's interface to LIBSVM. The iris data used here is provided in the base R installation, and can be loaded with the command `data(iris)`. We load the `e1071` library containing the SVM package with

```
> library(e1071)
```

and then we can split the data into a training and test set, and create an SVM model for the iris data as follows:

```
> data(iris)
> library(e1071)
> iristrain<-iris[c(1:30,51:80,101:130),]
> iristest<-iris[c(31:50,81:100,131:150),]
> irismodel<-svm(iristrain$Species~.,data=iristrain,
  scale = FALSE)
```

```
.*
```

```
optimization finished, #iter = 96
```

```
nu = 0.064753
```

```
obj = -2.011841, rho = 0.098439
```

```

nSV = 10, nBSV = 1
*
optimization finished, #iter = 41
nu = 0.060473
obj = -1.814402, rho = 0.205714
nSV = 9, nBSV = 0
*
optimization finished, #iter = 40
nu = 0.390381
obj = -16.413092, rho = 0.095269
nSV = 26, nBSV = 21
Total nSV = 36
>

```

Note that the output of the `svm` command is identical to that of the LIBSVM `svm-train` program. Print and summary methods for the resulting SVM model exist:

```
> summary(irismodel)
```

Call:

```
svm.formula(formula = iris.train$Species ~ .,
data = iris.train, scale = FALSE)
```

Parameters:

```

SVM-Type: C-classification
SVM-Kernel: radial
cost: 1
gamma: 0.25

```

Number of Support Vectors: 36

(5 16 15)

Number of Classes: 3

Levels:

setosa versicolor virginica

To test the model on the test set, we use

```
> predict.svm(irismodel, iristest)
```

```
[1] setosa      setosa      setosa      setosa
```

```
[5] setosa      setosa      setosa      setosa
```

...

...

```
[53] virginica  virginica  virginica  virginica
```

```
[57] virginica  virginica  virginica  virginica
```

Levels: setosa versicolor virginica

```
> table(predict.svm(irismodel, iristest),
  iristest$Species)
```

	setosa	versicolor	virginica
setosa	20	0	0
versicolor	0	19	1
virginica	0	1	19

We see that the R package provides us with the same functionality as the LIBSVM package, but with a more familiar interface — similarly for the MATLAB and S-PLUS packages. The use of these packages is then generally recommended for experimentation and implementation. However, those interested in theoretical development may wish to use the original package, particularly in cases where they desire to modify the underlying C code, either to obtain additional information not usually available, or to implement alternatives to the core SV technique.

As regards model selection, note that in the example above, we have used default values for our Gaussian kernel parameter γ , and regularization parameter C . In practice, cross-validation will be necessary for the choice of such parameters. Furthermore, although a number of well-known kernels are commonly used, particularly the Gaussian kernel, a *No Free Kernel* theorem exists, analogous to the No Free Lunch principle. This says that no kernel works well on all problems. Thus, it seems we should tailor our kernel to our problem. The technique for this we see exhibiting the most promise in future is the proposal in Cristianini, Kandola, Elisseeff and Shawe-Taylor (2000) of *kernel-target alignment*, although other techniques also exist (see e.g. Scholkopf, Simard, Smola and Vapnik (1998)).

The Gaussian kernel is commonly used because it traditionally displays good practical results, although not necessarily optimal. Furthermore, more complicated kernels although more suitable may reduce performance due to the higher variance incurred by estimating more parameters. A special interest in the Gaussian kernel has thus developed, leading to a recent paper by Sathiyaraj Keerthi and Chih-Jen Lin (co-author of LIBSVM) investigating the asymptotic behaviour of the Gaussian kernel SVM for various combinations of parameters. The results of this paper are useful, for they allow us easily to detect when over-

and underfitting are occurring. The authors show that if the cost parameter C becomes severely too small, underfitting occurs, with all points being assigned to the larger class, regardless of the kernel being used. This is usually detected when all training and validation points are classified to the same class. On the other hand, the Gaussian kernel can separate any consistent data set (i.e. no point is assigned to more than one class). This has the effect that if the cost parameter becomes big enough (relative to $\frac{1}{\gamma}$), the entire data set can be ‘learnt’, leading to overfitting — this manifests itself in an extremely good apparent error, but classifying all validation points to the majority class. This overfitting is most severe when γ is large, as this typically results in a very flexible classifier. The flexibility associated with a large value of γ means that the SVM is very sensitive to the correct choice of cost parameter. With smaller values of γ , this sensitivity decreases, and a suitable choice of cost parameter is often easier to find. For more details, and advice on using these results for more efficient model selection, see Keerthi and Lin (2003).

A common reason for poor SVM classifier performance through over- or underfitting is data not representative of the underlying population, or unbalanced data — cases where one of the classes is far more likely. In this second case, the SVM can usually obtain good performance by ignoring the class in the minority, and always predicting the larger class. If prediction of the smaller class is more important, we would typically want to emphasise it, typically by a higher cost parameter for misclassification of elements of the smaller class. This can be seen as a more complex cost function for penalizing inseparability, and this simple case can be implemented by the `-w1` option in LIBSVM or the `class.weights` parameter in R.

4. Summary

This paper discusses SVM packages and the practical aspects of applying SVMs.

A variety of SVM packages are available, both stand-alone, and as libraries for those planning on utilising the code in their own software. Techniques have been developed to reduce the computational complexity of the basic SVM formulation, but since these techniques are relatively new, they are not yet routinely implemented in all packages. Practitioners working with large data sets (many thousands of data points) should thus make sure that the package they are considering is capable of handling such large data sets reliably.

SVMs have exhibited excellent performance on many benchmark and practical problems, and are fast becoming a popular method in the machine learning toolbox. However, despite its good performance, experience has shown that many simpler techniques may match its performance. The SVM should thus not be considered a global solution, but only another viable alternative to existing techniques.

Acknowledgements

The authors would like to thank Prof. Sarel Steel, as well as the two anonymous referees, for useful comments and suggestions. We would also like to thank the administrators of the UCI data repository.

References

- BLAKE, C. and MERZ, C. (1998). UCI repository of machine learning databases. <http://www.ics.uci.edu/~mllearn/MLRepository.html>
- BOSER, B.E., GUYON, I.M. and VAPNIK, V.N. (1992). A training algorithm for optimal margin classifiers. D. Haussler (editor) *Proceedings of the Fifth*

- Annual Workshop on Computational Learning Theory*, pp. 144–152. ACM Press, Pittsburg, PA.
- CHANG, C.-C. and LIN, C.-J. (2000). *LIBSVM: a Library for Support Vector Machines (Version 2.2)*. National Taiwan University, Taipei. <http://www.csie.ntu.edu.tw/~cjlin/libsvm/>
- COLLOBERT, R. and BENGIO, S. (2000). Support vector machines for large-scale regression problems. IDIAP Research Report 00-17, Dalle Molle Institute for Perceptual Artificial Intelligence (IDIAP), Martigny, Switzerland.
- CHRISTIANINI, N., KANDOLA, J., ELISEEFF, A. and SHAW-TAYLOR, J. (2002). On kernel target alignment. *Journal of Machine Learning Research*. Submitted.
- GUNN, S.R. (1998). Support vector machines for classification and regression. Tech. rep., Department of Electronics and Computer Science, Faculty of Engineering and Applied Science, University of Southampton.
- JOACHIMS, T. (1997). Text categorization with support vector machines: Learning with many relevant features. LS VIII-Report 23, Fachbereich Informatik, Universität Dortmund, Dortmund, Germany. Revised April 1998.
- JOACHIMS, T. (1998). Making large-scale SVM learning practical. LS VIII-Report 24, Fachbereich Informatik, Universität Dortmund, Dortmund, Germany. http://www.-ai.cs.uni-dortmund.de/svm_light
- KEERTHI, S.S. and LIN, C.J. (2003). Asymptotic behaviors of support vector machines with Gaussian kernel. Manuscript Number 2621.
- LEE, J. H. and LIN, C.J. (2000). Automatic model selection for support vector machines. Tech. rep., Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan.
- SCHÖLKOPF, B., SIMARD, P., SMOLA, A.J. and VAPNIK, V.N. (1998). Prior knowledge in support vector kernels. M.I. Jordan, M.J. Kearns and S. Solla (editors) *Advances in Neural Information Processings Systems, Volume 10*, pp. 640–646. MIT Press, Cambridge, MA.
- STEEL, S.J. (2003). Personal Communication.
- Manuscript received, 2003.11, revised, 2004.03, accepted, 2004.05.